

Mini Transaction Ledger

Architecture, Internals & Docker Setup

Kazi Tasrif

September 7, 2026

This document explains how the Mini Transaction Ledger is put together: how the frontend and backend talk to each other, the database schema, how authentication works, what each core feature does, and how the whole system is packaged with Docker.

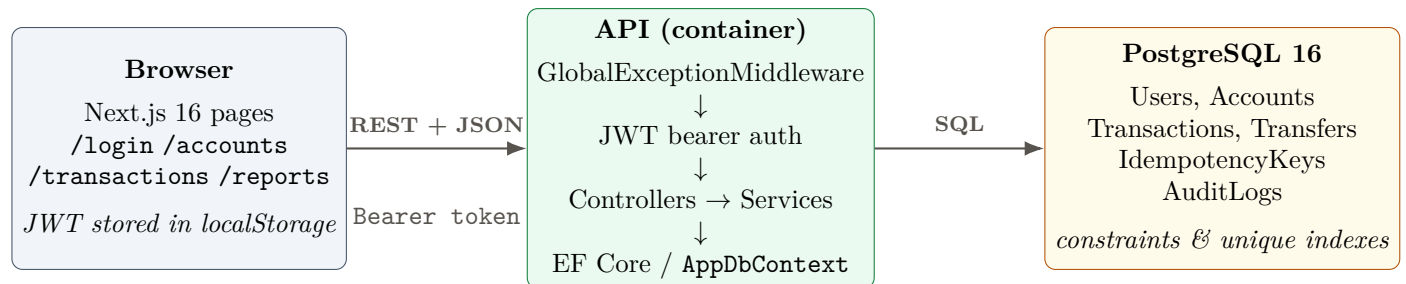
Stack. ASP.NET Core 9 (backend) • PostgreSQL 16 (database) • Next.js 16 + TypeScript (frontend) • xUnit + Testcontainers (tests) • Docker Compose (packaging).

Contents

1	Architecture: how the frontend and backend interact	2
2	Database schema	2
3	Key code components	3
3.1	Backend — backend/TransactionLedger/	3
3.2	Frontend — frontend/	3
4	API endpoints	4
5	Core features	4
5.1	Accounts	4
5.2	Transactions	4
5.3	Reversals	5
5.4	Transfers and row-level locking	5
5.5	Reports	5
5.6	Authentication (JWT)	6
5.7	Idempotency	6
5.8	Errors: one shape, one stable identifier	7
6	Docker setup	7

1 Architecture: how the frontend and backend interact

The system is a plain three-tier REST application. The frontend never touches PostgreSQL directly; every value on screen arrives over `fetch()` as JSON. The backend owns every business rule; the database owns integrity constraints as a second, independent line of defence.

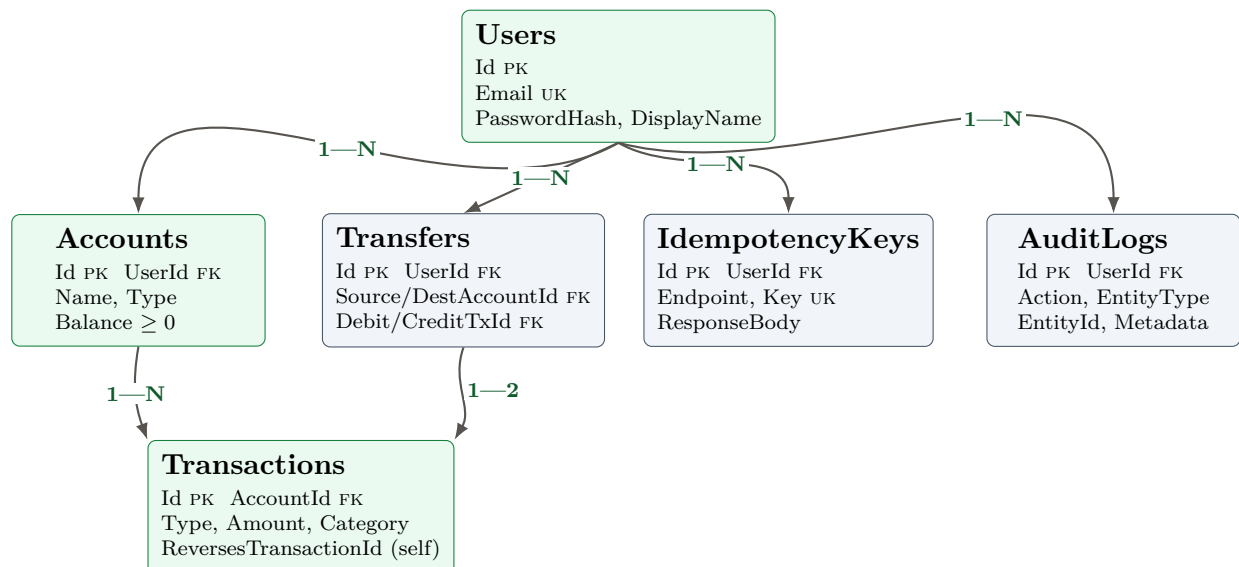


CORS. Frontend and backend are different origins in development, so the API's CORS policy allows exactly the frontend's origin:

- GET and POST only — there is no PUT, PATCH, or DELETE anywhere in this API. The ledger is append-only: a correction is always a new row, never an edit.
- The custom `Idempotency-Key` and `Idempotent-Replay` headers are explicitly allowed and exposed — custom headers are not on the browser's default CORS safelist.

2 Database schema

Six tables. Money and identifiers use `decimal` / `uuid` throughout — never floating point.



- **1—N** — one row on the arrow's start relates to many rows at its end (every FK in the table above).
- **1—2** — one transfer always relates to exactly two transactions: its debit leg and its credit leg.
- Not shown: a transaction may optionally reverse exactly one other transaction — a self-referencing,

one-to-one-or-none link (`ReversesTransactionId`, unique when set).

Constraints enforced by the database itself

- Unique email per user; unique account name per user (case-insensitive).
- Account balance can never go negative (`CHECK` constraint).
- Transaction amount is always positive, capped at a maximum.
- A transaction can be reversed at most once (partial unique index).
- A transfer's source and destination accounts must differ.
- An idempotency key is unique per user and endpoint — the entire retry-safety mechanism (§5.7).

3 Key code components

3.1 Backend — `backend/TransactionLedger/`

Folder	Purpose
<code>Controllers/</code>	HTTP surface only: validate the request shape, call one service method, map the result to a DTO.
<code>Services/</code>	Every business rule, one service per aggregate — <code>AccountService</code> , <code>TransactionService</code> , <code>TransferService</code> , <code>ReportService</code> , <code>IdempotencyService</code> , <code>AuditService</code> .
<code>Domain/</code>	Entities and enums: the shape the database mirrors.
<code>DTOs/</code>	One request/response type per endpoint. An EF entity is never serialised directly over HTTP.
<code>Data/</code>	<code>AppDbContext</code> , entity configurations, EF Core migrations.
<code>Middleware/</code>	<code>GlobalExceptionHandler</code> — the single place an exception becomes a <code>problem+json</code> response with a stable error code and a <code>traceId</code> .
<code>Configuration/</code>	Strongly-typed, validated-at-startup options (<code>JwtSettings</code> , <code>DatabaseSettings</code> , <code>CorsSettings</code>).

No repository layer: EF Core's `DbSet<T>` already is one, and a hand-rolled wrapper around it would add indirection without removing duplication.

3.2 Frontend — `frontend/`

Folder	Purpose
<code>app/</code>	Next.js App Router pages. (<code>app/</code>) is a route group for every authenticated screen, sharing one sidebar layout and one route guard.
<code>components/</code>	<code>ui/</code> — shared primitives (<code>Button</code> , <code>Input</code> , <code>Drawer</code> , <code>Card</code>). Feature folders compose them, one concern each.
<code>lib/</code>	<code>api.ts</code> (every HTTP call), <code>auth.tsx</code> (session + route guard), <code>errors.ts</code> (the error-code switch), <code>useResource.ts</code> (fetch-with-abort-and-status hook).
<code>types/</code>	Hand-written TypeScript mirrors of the backend's DTOs.

No Redux, no React Query: `useResource` plus React's own `useState` covers every screen.

4 API endpoints

Swagger UI is available at `/swagger` while the API runs in Development, with a working **Authorize** button for a bearer token from `/api/auth/login`.

Method	Path	Idempotency
POST	<code>/api/auth/register</code> , <code>/api/auth/login</code>	—
GET POST	<code>/api/accounts</code>	—
GET	<code>/api/accounts/{id}</code> , <code>/api/accounts/{id}/balance</code>	—
GET POST	<code>/api/accounts/{id}/transactions</code>	required on POST
GET	<code>/api/transactions/{id}</code>	—
POST	<code>/api/transactions/{id}/reverse</code>	by construction
GET POST	<code>/api/transfers</code>	required on POST
GET	<code>/api/reports/{summary, categories, monthly}</code>	—

- Every path except `/api/auth/*` requires Authorization: `Bearer <token>`.
- **required on POST** — the two money-moving endpoints reject a request with no Idempotency-Key header.
- **by construction** — reversal needs no key of its own: the unique index on `ReversesTransactionId` already blocks a transaction from being reversed twice.

5 Core features

5.1 Accounts

- Every account starts at balance 0.00 — there is no opening-balance field.
- Types: Cash, Savings, Business, Wallet, Other.
- Name is unique per user, case-insensitive.
- Funded only by recording a Credit transaction against it.
- Balance can never go negative — enforced by both the service layer and a database `CHECK` constraint.

5.2 Transactions

- Two types: **Credit** (money in) or **Debit** (money out).
- Amount is always stored positive; direction comes from the type, never from the sign.
- Category is a fixed set: Salary, Food, Transport, Bills, Shopping, Entertainment, Health, Other, plus two system-only categories (Transfer, Reversal).
- Never updated or deleted — the ledger is append-only.
- A debit that would take the balance below zero is rejected; the balance is left unchanged.

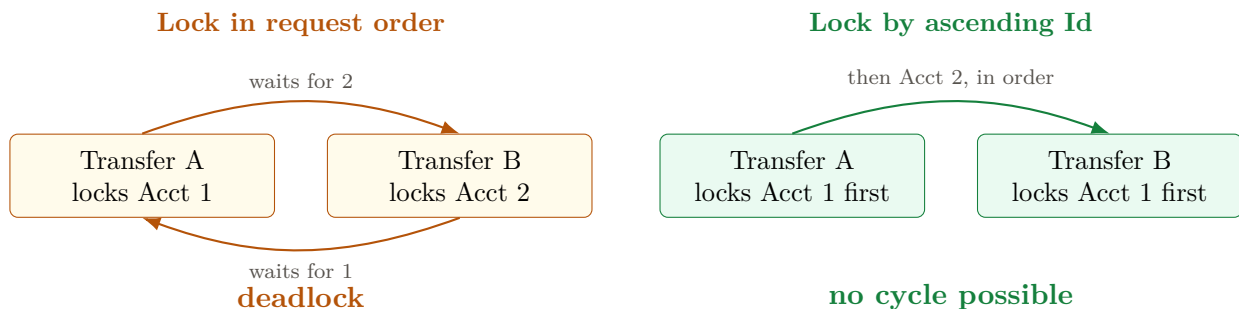
5.3 Reversals

- The only way to correct a mistake — no edit, no delete.
- Creates a new, opposite transaction; the original stays untouched and is simply marked *reversed*.
- A transaction can be reversed at most once, enforced by a unique database index.
- Transfers and reversal transactions themselves cannot be reversed.

5.4 Transfers and row-level locking

- Moves money between two accounts owned by the same user.
- Built from two linked transactions — one debit, one credit — inserted inside a single database transaction.
- Source and destination accounts must differ.
- Both accounts are locked together, in one statement, in ascending Id order — always the same order, system-wide.

Locking in whichever order a request happens to name “source” and “destination” is how two simultaneous, opposite transfers deadlock each other:

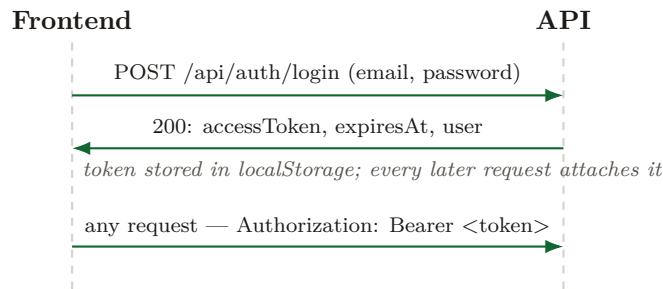


- `SELECT ... WHERE Id = ANY(@ids) ORDER BY Id FOR UPDATE` locks both rows in one statement.
- The ownership check is inside the same query: a foreign or missing account returns zero rows — a 404, not a runtime error.
- Both transactions, the transfer record, and both balance updates commit together, or not at all.

5.5 Reports

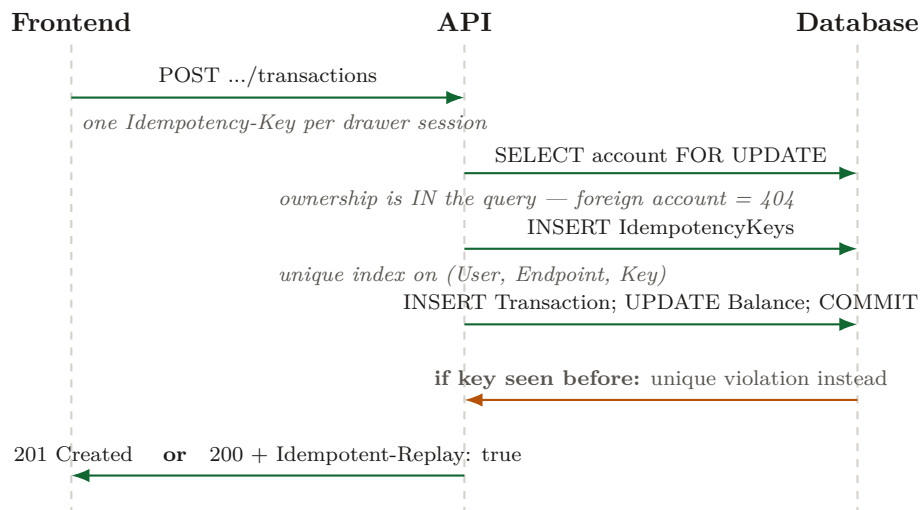
- **Summary** — total balance, this month’s credits, debits, net change, and account count.
- **Category breakdown** — spending by category, excluding transfers and reversed transactions.
- **Monthly trend** — credits vs. debits per calendar month.
- All aggregation runs in SQL — nothing is summed in application code.

5.6 Authentication (JWT)



- Login exchanges email and password for a signed JSON Web Token, valid **60 minutes**.
- Signed with HMAC-SHA256 using a server-side key; there is **no server-side session store**.
- The token carries the user's id (sub claim), email, issued time, and expiry — nothing else.
- Every protected request must include `Authorization: Bearer <token>`.
- The API validates signature, issuer, audience, and expiry on every request — zero clock skew allowed.
- User identity always comes from the token's own sub claim. No endpoint accepts a client-supplied `userId`.
- No refresh token: an expired token requires signing in again.

5.7 Idempotency



- Every `POST` that moves money requires an `Idempotency-Key` header.
- The key is written to a unique index on `(UserId, Endpoint, Key)`, inside the *same* database transaction as the financial work.
- A retried request with the same key hits the unique violation and replays the original stored response — the money moves only once.

- No in-memory flag, no distributed lock, no timeout: the unique index is the entire mechanism.

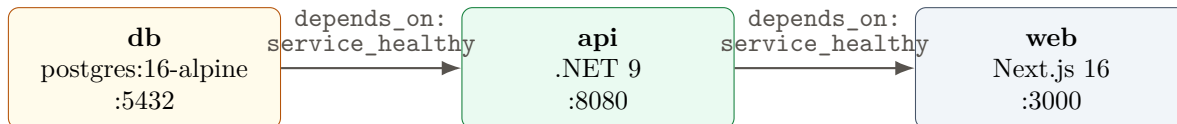
5.8 Errors: one shape, one stable identifier

Every failure — validation, ownership, business-rule rejection, or a genuine bug — leaves the API as `application/problem+json` with a `code` field:

```
{ "type": "...", "status": 400, "code": "AMOUNT_NOT_POSITIVE", "detail": "..." }
```

- The frontend switches on `code`, **never** on `title` or `detail`.
- `title/detail` are for a human to read; `code` is for code to read, and can be reworded later without breaking anything.

6 Docker setup



- `docker compose up -build` builds and starts all three containers.
- Plain `depends_on` only waits for a container to *start*, not for Postgres to accept connections, so every dependency above uses `condition: service_healthy` instead — `pg_isready` for the database, `curl -f .../health` for the API.
- **Backend image:** two-stage build — `dotnet publish` on the full SDK image, then only the compiled output copied into the smaller `aspnet` runtime image. `.csproj` files are restored before the rest of the source, so the layer cache only re-runs `dotnet restore` when a dependency actually changes.
- **Frontend image:** three-stage build — install dependencies, compile, then Next.js's `standalone` output, which ships only the dependencies actually used at runtime.
- **Build-time vs. run-time config:** anything prefixed `NEXT_PUBLIC_` is inlined into the browser bundle at build time, not read at container start — it must arrive as a build `ARG`, not an `environment:` variable.
- **Two addresses for the same API:** inside the Compose network, the frontend's server-side code reaches the API at `http://api:8080`; the browser reaches it at `http://localhost:8080` instead, since the Docker service name `api` is not resolvable outside the container network.
- **Secrets:** a single gitignored `.env` file supplies the database password and signing key as environment variables at container start — nothing is baked into an image.
- **Migrations:** applied automatically on startup, guarded by a config flag Compose sets to `true` — so `docker compose up -build` produces a fully migrated, working database with no second command.